



西南交通大学
Southwest Jiaotong University

云计算技术课程设计报告

基于华为云 CCE 的 Kubernetes 平台搭建
与 Spark 大数据分析

课 程 云计算技术

学 院 计算机与人工智能学院

专 业 计算机科学与技术

班 级 计算机科学与技术 04 班

学 号 2023112594

姓 名 陶秋源

2026 年 6 月

目 录

第 1 章	华为云环境信息	1
1.1	设计目标与总体架构	1
1.2	华为云服务与账号信息	1
1.3	CCE 集群与节点规格	2
第 2 章	第一部分：云计算平台搭建	3
2.1	任务 1：应用容器化	3
2.1.1	后端镜像（多阶段构建 + 自选包）	3
2.1.2	前端镜像（含学号姓名）	3
2.1.3	本地联调与推送 SWR	4
2.2	任务 2：CCE 集群搭建	5
2.3	任务 3：应用部署	5
2.4	任务 4：持久化存储	6
2.5	任务 5：ConfigMap Volume 挂载	7
2.5.1	Volume 挂载与 envFrom 两种方式的适用场景	8
2.6	任务 6：HPA 弹性伸缩	8
2.6.1	结果分析：扩容延迟、冷却时间与降本价值	10
第 3 章	第二部分：Spark 大数据分析	12
3.1	A-0：环境部署	12
3.1.1	Spark Operator 安装	12
3.1.2	关键问题：Executor 因 CPU 请求不足无法调度	12
3.1.3	WordCount 验证	12
3.2	A-1：数据清洗	13
3.3	A-2：Spark SQL 统计分析	14
3.3.1	Q1：各电影类型数量与平均评分（GROUP BY）	14
3.3.2	Q2：高分电影 Top10（ORDER BY Top-N）	14
3.3.3	Q3：各年代电影数量与平均评分趋势（时间维度）	15
3.3.4	Q4：每个类型评分最高的电影（窗口函数）	15
3.3.5	Q5：高于本年代平均分的经典电影（JOIN）	16
3.4	A-3：性能对比与 Amdahl 分析	16

目 录	II
第 4 章 附加题	19
4.1 附加题 1: 监控系统 Prometheus + Grafana	19
4.1.1 部署	19
4.1.2 Grafana Dashboard	19
4.1.3 Prometheus Pull 采集原理与指标含义	19
4.2 附加题 2: CI/CD 流水线	20
4.2.1 流水线设计	20
4.2.2 持续集成 vs 持续部署与 GitOps	21
4.3 附加题 3: 分布式 AI 训练 (PyTorch DDP)	22
4.3.1 实验设计	22
4.3.2 结果对比	22
4.3.3 AllReduce 梯度同步机制	23
4.3.4 数据并行 vs 模型并行	23
4.3.5 批判性思考	23
第 5 章 总结与收获	25
第 6 章 附录	26
6.1 代码仓库	26
6.2 关键资源清单	26
6.3 镜像清单 (SWR group17)	26

第 1 章 华为云环境信息

1.1 设计目标与总体架构

本课程设计基于华为云搭建一套完整的云原生平台：以云容器引擎 CCE 托管的 Kubernetes 集群为核心，部署「Flask 后端 API + Redis 数据库」两层 Web 应用，依次实现容器化、对外暴露、配置分离、数据持久化与弹性伸缩；随后在同一集群上通过 Spark Operator 运行 PySpark 大数据分析作业；最后扩展完成监控系统、CI/CD 流水线与分布式 AI 训练三个附加专题。总体技术架构如图 1-1 所示。

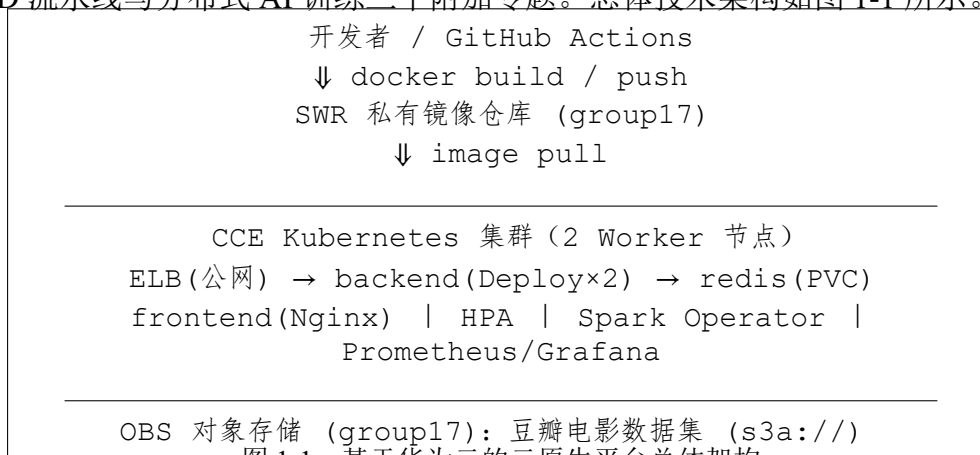


图 1-1 基于华为云的云原生平台总体架构

1.2 华为云服务与账号信息

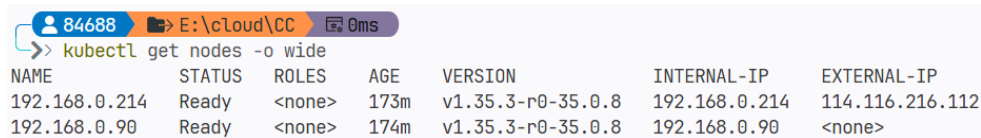
本设计使用了表 1-1 所列的华为云服务，区域（Region）统一为 华北-北京四（cn-north-4）。

表 1-1 华为云服务与命名

服务	作用	本设计取值
CCE	托管 Kubernetes 集群	集群 group17，版本 v1.35.3
SWR	私有镜像仓库	组织 group17
OBS	对象存储（数据集）	桶 group17（s3a 接口）
ELB	弹性负载均衡（公网入口）	自动创建，公网 IP 1.94.238.112

1.3 CCE 集群与节点规格

集群由华为云托管 Master 节点（免运维），学生创建并管理 2 个 Worker 节点，节点规格为 **c9.large.4**（2 vCPU / 8 GiB），操作系统为 Huawei Cloud EulerOS 2.0，容器运行时为 containerd 2.2.1，Kubernetes 版本 v1.35.3（≥ 1.27，满足要求）。节点信息如图 1-2（任务 2 验收截图）所示。



NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP
192.168.0.214	Ready	<none>	173m	v1.35.3-r0-35.0.8	192.168.0.214	114.116.216.112
192.168.0.90	Ready	<none>	174m	v1.35.3-r0-35.0.8	192.168.0.90	<none>

图 1-2 kubectl get nodes -o wide: 2 个 Worker 节点 Ready，版本 v1.35.3

为支撑后续 HPA（任务 6）与监控，集群额外部署了 metrics-server（提供 metrics.k8s.io 资源指标 API），kubectl top nodes 可正常读取节点 CPU/内存用量。

第2章 第一部分：云计算平台搭建

2.1 任务1：应用容器化

2.1.1 后端镜像（多阶段构建 + 自选包）

后端为 Flask API，提供 `/api/ping`（健康检查）、`/api/visits`（写 Redis 计数，演示三层通信）、`/api/info`、`/api/compute`（CPU 压测入口）四个接口。Dockerfile `.backend` 采用两阶段构建：第一阶段将依赖安装到独立目录，第二阶段仅复制依赖与代码，避免把 `pip`/编译工具带入运行镜像，显著减小体积。在 `requirements.txt` 中加入了自选包 `requests`。

```
1 # Stage 1: 安装依赖
2 FROM python:3.11-slim AS builder
3 WORKDIR /build
4 COPY requirements.txt .
5 RUN pip install --no-cache-dir -r requirements.txt --target /build/
   packages
6 # Stage 2: 运行时镜像（更小）
7 FROM python:3.11-slim
8 WORKDIR /app
9 COPY --from=builder /build/packages /app/packages
10 COPY . .
11 ENV PYTHONPATH=/app/packages
12 EXPOSE 5000
13 CMD ["python", "app.py"]
```

Listing 2.1 Dockerfile.backend（多阶段构建）

2.1.2 前端镜像（含学号姓名）

前端为 Nginx 静态页，`static/index.html` 中加入了本人学号与姓名用于答辩识别：

```
1 <p class="info">学号：2023112594 姓名：陶秋源 班级：计算机科学与技术04班
   </p>
```

Listing 2.2 index.html 关键行

2.1.3 本地联调与推送 SWR

本地执行 `docker compose up` 启动 `backend`、`frontend`、`redis` 三个容器，浏览器访问前端页面（图 2-1）点击按钮触发请求，后端日志打印 收到请求 `GET /api/ping`、`/api/visits` 等记录（图 2-2），证明「前端 Nginx → 后端 Flask → Redis」三层通信正常。



图 2-1 前端页面（含学号姓名）调用后端接口返回 JSON



图 2-2 docker compose 后端日志显示收到请求

镜像推送到华为云 SWR 时需注意：Docker 29.x 默认生成 OCI 格式 manifest，而 SWR 仅支持经典 Docker 格式，故构建时使用 `--provenance=false`。推送命令如下，推送后在 SWR 控制台可见 `backend` 与 `frontend` 镜像及其 Tag（图 2-3）。

```
1 docker login -u cn-north-4@<AK> -p <SWR 登录 密钥> swr.cn-north-4.  
   myhuaweicloud.com  
2 docker tag backend:v1 swr.cn-north-4.myhuaweicloud.com/group17/backend:v1
```

```
3 docker push swr.cn-north-4.myhuaweicloud.com/group17/backend:v1
```

Listing 2.3 推送镜像到 SWR group17

☐ 镜像名称	所属组织	类型	版本数	更新时间	操作
☐ metrics-server	group17	私有	1	2026/06/10 18:35:26 GMT+08:00	镜像自动同步
☐ spark-operator	group17	私有	1	2026/06/10 18:16:57 GMT+08:00	镜像自动同步
☐ prometheus-operator	group17	私有	1	2026/06/10 18:16:55 GMT+08:00	镜像自动同步
☐ prometheus-config-reloader	group17	私有	1	2026/06/10 18:16:55 GMT+08:00	镜像自动同步
☐ node-exporter	group17	私有	1	2026/06/10 18:16:54 GMT+08:00	镜像自动同步
☐ alertmanager	group17	私有	1	2026/06/10 18:16:53 GMT+08:00	镜像自动同步
☐ prometheus	group17	私有	1	2026/06/10 18:16:52 GMT+08:00	镜像自动同步
☐ grafana	group17	私有	1	2026/06/10 18:16:51 GMT+08:00	镜像自动同步
☐ kube-webhook-certgen	group17	私有	1	2026/06/10 18:16:50 GMT+08:00	镜像自动同步
☐ pyspark	group17	私有	1	2026/06/10 18:16:49 GMT+08:00	镜像自动同步
☐ redis	group17	私有	1	2026/06/10 18:16:48 GMT+08:00	镜像自动同步
☐ frontend	group17	私有	1	2026/06/10 18:16:46 GMT+08:00	镜像自动同步
☐ pytorch-ddp	group17	私有	1	2026/06/10 18:16:44 GMT+08:00	镜像自动同步
☐ backend	group17	私有	1	2026/06/10 18:15:57 GMT+08:00	镜像自动同步

图 2-3 SWR 控制台 group17 组织下的镜像列表（含名称与 Tag）

2.2 任务 2: CCE 集群搭建

集群创建与节点规格已在第 1 章描述，2 个 Worker 节点均为 Ready、版本 v1.35.3（≥ 1.27），验收截图见图 1-2。KubeConfig 从 CCE 控制台下载，本地配置 kubectl 使用其中的 external 上下文（公网 API 端点）直接操作集群。

2.3 任务 3: 应用部署

参考附录 A-2 模板，部署了 6 类资源：后端 Deployment（副本 2、含 resources 限制、envFrom 注入 ConfigMap、secretKeyRef 注入 Redis 密码）、Redis Deployment（副本 1、limits.memory=256Mi≤512Mi）、后端 LoadBalancer Service（注解 kubernetes.io/elb.class: union + elb.autocreate 自动创建公网 ELB）、Redis ClusterIP Service、ConfigMap（REDIS_HOST=redis-svc）、Secret（密码 base64 编码，无明文）。

```
1 spec:
2   replicas: 2
```



```

3  template:
4    spec:
5      containers:
6      - name: backend
7        image: swr.cn-north-4.myhuaweicloud.com/group17/backend:v1
8        resources:
9          requests: { cpu: "100m", memory: "128Mi" }
10         limits:   { cpu: "500m", memory: "512Mi" }
11        envFrom:
12        - configMapRef: { name: backend-config } # 注入 Redis 地址
13        env:
14        - name: REDIS_PASSWORD
15          valueFrom:
16            secretKeyRef: { name: redis-secret, key: password } # 注入密码

```

Listing 2.4 后端 Deployment 关键片段

部署后所有 Pod 均为 Running (图 2-4), 后端 Service 自动绑定华为云公网 ELB, 分配公网 IP **1.94.238.112** (图 2-5)。浏览器访问 <http://1.94.238.112/api/ping> 返回 `{"status":"ok"}` (图 2-6); 连续访问 `/api/visits` 计数依次递增且 `served_by` 在两个后端 Pod 间轮换, 验证了 ELB 负载均衡、ConfigMap/Secret 注入与三层通信全部正常。ConfigMap 与 Secret 内容见图 2-7, Secret 的 `password` 字段为 base64 (`Z3JvdXAxNy4=`), 无明文。

```

84688 E:\cCloud\CC 1ms
>> kubectl get pods -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
backend-8697c9f599-pklpl	1/1	Running	0	128m	10.0.0.38	192.168.0.214	<none>	<none>
backend-8697c9f599-sv6pj	1/1	Running	0	128m	10.0.0.21	192.168.0.90	<none>	<none>
frontend-6cf9b7d745-zr4xf	1/1	Running	0	122m	10.0.0.22	192.168.0.90	<none>	<none>
redis-7466fd7f75-c9klg	1/1	Running	0	122m	10.0.0.39	192.168.0.214	<none>	<none>

图 2-4 kubectl get pods: backend×2、redis、frontend 全部 Running

```

84688 E:\cCloud\CC 0ms
>> kubectl get svc backend-svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
backend-svc	LoadBalancer	10.247.26.27	1.94.238.112,192.168.0.216	80:31878/TCP	129m

图 2-5 后端 Service 为 LoadBalancer, 公网 IP 1.94.238.112

2.4 任务 4: 持久化存储

为 Redis 配置 PVC (`storageClassName: csi-disk`, 即华为云 EVS 云硬盘, 申请 10 Gi), 并修改 Redis Deployment 将 `/data` 挂载到该 PVC; 同时 Redis 以 `--appendonly yes` 开启 AOF 持久化。由于 EVS 为 `ReadWriteOnce`, Deployment 更新策略设为 `Recreate` 以避免新旧 Pod 同时挂载同一磁盘。

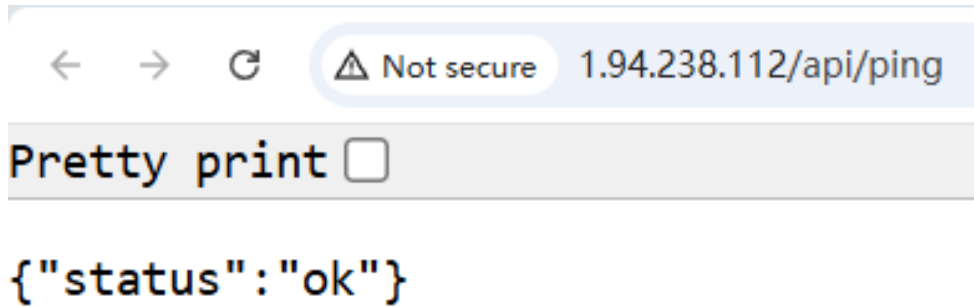


图 2-6 浏览器访问 ELB 公网 IP, /api/ping 返回 status:ok



图 2-7 ConfigMap 注入 Redis 地址; Secret 密码 base64 无明文

PVC 创建后状态为 **Bound** (图 2-8)。持久化验证流程: 向 Redis 写入 SET testkey "hello" (图 2-9) → kubectl delete pod 删除 Redis Pod 触发重建 (图 2-10) → 新 Pod Running 后再次 GET testkey 仍返回 "hello" (图 2-11), 证明数据随云硬盘持久化, 未随 Pod 销毁而丢失。

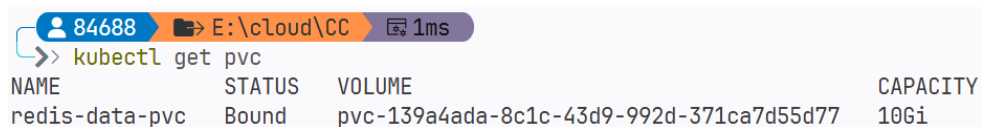


图 2-8 kubectl get pvc: redis-data-pvc 状态 Bound (10Gi csi-disk)

2.5 任务 5: ConfigMap Volume 挂载

将前端 Nginx 的反向代理配置 (default.conf) 改为以 ConfigMap 的 Volume 形式挂载到 /etc/nginx/conf.d/default.conf (subPath 挂载), 其中 upstream 指向后端 Service backend-internal:5000。

随后将 ConfigMap 中后端端口由 5000 改为 5001 并 kubectl apply。注意 Kubernetes 已知限制: 使用 subPath 挂载时 ConfigMap 更新不会自动同步到 Pod 内, 故需删除前端 Pod 让 Deployment 重建。重建后 exec 进入前端 Pod 执行 cat default

```

84688 E:\cloud\CC 0ms
>> $rp = kubectl get pod -l app=redis -o jsonpath='{.items[0].metadata.name}'
84688 E:\cloud\CC 200ms
>> kubectl exec $rp -- redis-cli -a 'group17.' --no-auth-warning SET testkey "hello"
OK
84688 E:\cloud\CC 355ms
>> kubectl exec $rp -- redis-cli -a 'group17.' --no-auth-warning GET testkey
hello

```

图 2-9 写入测试数据 SET testkey hello

```

84688 E:\cloud\CC 335ms
>> kubectl delete pod $rp
pod "redis-7466fd7f75-c9klg" deleted from default namespace
84688 E:\cloud\CC 746ms
>> kubectl get pods -l app=redis
NAME                                READY   STATUS    RESTARTS   AGE
redis-7466fd7f75-rvkbq             1/1     Running   0           6s

```

图 2-10 删除 Redis Pod，触发自动重建

.conf，可见端口已变为 5001（图 2-13），证明挂载文件随 ConfigMap 更新而生效。

2.5.1 Volume 挂载与 envFrom 两种方式的适用场景

envFrom（环境变量注入）适合少量、扁平的 **key-value 配置**（如开关、地址、端口），应用通过 `os.environ` 读取，简单直接，但环境变量在容器启动时固定，更新需重建 Pod，且不适合大段结构化文本。**Volume 挂载**则适合**完整配置文件**（如 `nginx.conf`、`application.yaml`）：以文件形式呈现，保留原有格式，且在非 `subPath` 挂载时支持 kubelet 周期性热更新（约分钟级）。本设计后端的 Redis 地址用 `envFrom` 注入，前端的整段 Nginx 配置用 Volume 挂载，正是按“短变量用环境变量、长文件用卷”的原则选择。

2.6 任务 6: HPA 弹性伸缩

确认 `kubectl top nodes` 有数据后（`metrics-server` 就绪），为后端创建 HPA：`minReplicas=1`、`maxReplicas=4`、CPU 目标利用率 60%。随后用压测脚本对 ELB 公网 IP 的 CPU 密集端点 `/api/compute` 发起约 120 并发的持续请求。

压测开始后约 1–2 分钟，`kubectl get pods -w` 监控窗口可见后端 Pod 数量从

```

84688 E:\cloud\CC 0ms
>> $rp2 = kubectl get pod -l app=redis -o jsonpath='{.items[0].metadata.name}'
84688 E:\cloud\CC 216ms
>> kubectl exec $rp2 -- redis-cli -a 'group17.' --no-auth-warning GET testkey
hello

```

图 2-11 重建后 GET testkey 仍返回 hello（数据持久化）

```

84688 E:\cloud\CC 0ms
>> $fp = kubectl get pod -l app=frontend -o jsonpath='{.items[0].metadata.name}'
84688 E:\cloud\CC 230ms
>> kubectl exec $fp -- cat /etc/nginx/conf.d/default.conf
server {
    listen 80;
    server_name _;

    # 静态前端页面 (含学号姓名)
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }

    # 反向代理到后端 Service (任务5: upstream 后端地址)
    location /api/ {
        proxy_pass http://backend-internal:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

```

图 2-12 修改前: 挂载配置 proxy_pass 指向:5000

```

84688 E:\cloud\CC 0ms
>> kubectl apply -f E:\cloud\CC\k8s\10b-frontend-nginx-configmap-5001.yaml
configmap/frontend-nginx-conf configured
84688 E:\cloud\CC 373ms
>> kubectl delete pod $fp
pod "frontend-6cf9b7d745-zr4xf" deleted from default namespace
84688 E:\cloud\CC 728ms
>> $fp2 = kubectl get pod -l app=frontend -o jsonpath='{.items[0].metadata.name}'
84688 E:\cloud\CC 203ms
>> kubectl exec $fp2 -- cat /etc/nginx/conf.d/default.conf
server {
    listen 80;
    server_name _;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }

    # 后端端口已从 5000 修改为 5001 (演示 ConfigMap 修改后挂载文件同步更新)
    location /api/ {
        proxy_pass http://backend-internal:5001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

```

图 2-13 修改并重建后: 挂载配置已更新为:5001

```
84688 E:\cloud 1ms
>> kubectl top pods -l app=backend
NAME                                CPU(cores)   MEMORY(bytes)
backend-8697c9f599-dr5rc            204m         27Mi
84688 E:\cloud 267ms
>> kubectl top pods -l app=backend
NAME                                CPU(cores)   MEMORY(bytes)
backend-8697c9f599-dr5rc            500m         27Mi
84688 E:\cloud 295ms
>> kubectl top pods -l app=backend
NAME                                CPU(cores)   MEMORY(bytes)
backend-8697c9f599-dr5rc            271m         27Mi
backend-8697c9f599-qgnsk            295m         28Mi
```

图 2-14 kubectl top nodes: metrics-server 提供资源指标

```
84688 E:\cloud 2ms
>> kubectl get hpa backend-hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 204%/60%  1        4        1         4m5s
84688 E:\cloud 179ms
>> kubectl get hpa backend-hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 204%/60%  1        4        1         4m12s
84688 E:\cloud 197ms
>> kubectl get hpa backend-hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 500%/60%  1        4        4         4m21s
```

图 2-15 创建 HPA: min1/max4/CPU 60%

1 增加到 2 乃至更多（图 2-16）；停止压测约 5 分钟后，Pod 数缩回 1（图 2-17）。

```
84688 E:\cloud 0ms
>> kubectl get pods -l app=backend -w
NAME                                READY   STATUS    RESTARTS   AGE
backend-8697c9f599-dr5rc            1/1     Running   0           11m
backend-8697c9f599-qgnsk            0/1     Pending   0           0s
backend-8697c9f599-qgnsk            0/1     Pending   0           0s
backend-8697c9f599-dbk8            0/1     Pending   0           0s
backend-8697c9f599-rczd            0/1     Pending   0           0s
backend-8697c9f599-qgnsk            0/1     ContainerCreating   0           0s
backend-8697c9f599-dbk8            0/1     Pending   0           0s
backend-8697c9f599-rczd            0/1     Pending   0           0s
backend-8697c9f599-qgnsk            0/1     Running   0           1s
backend-8697c9f599-dr5rc            0/1     Running   0           13m
backend-8697c9f599-qgnsk            1/1     Running   0           12s
backend-8697c9f599-dr5rc            1/1     Running   0           13m
```

图 2-16 压测中后端 Pod 从 1 扩容到多个

2.6.1 结果分析：扩容延迟、冷却时间与降本价值

①扩容延迟的原因。从 CPU 飙升到新 Pod 就绪存在分钟级延迟，源于多段串联：metrics-server 默认每 15 s 采集一次指标、HPA 控制器默认每 15 s 评估一次、计算出目标副本后还需经历调度、镜像拉取、容器启动与就绪探针通过。这些周期叠加导致扩容并非”瞬时”。

```
84688 E:\cloud 0ms
>> kubectl get pods -l app=backend -w
backend-8697c9f599-dr5rc 1/1 Running 0 13m
backend-8697c9f599-rczwd 0/1 Terminating 0 10m
backend-8697c9f599-dbk8 0/1 Terminating 0 10m
backend-8697c9f599-qgnsk 1/1 Terminating 0 10m
backend-8697c9f599-rczwd 0/1 Terminating 0 10m
backend-8697c9f599-dbk8 0/1 Terminating 0 10m
backend-8697c9f599-qgnsk 1/1 Terminating 0 10m
```

图 2-17 停止压测后 Pod 数缩回 1

②冷却时间（稳定窗口）的意义。HPA 缩容默认有 300s 的稳定窗口 (stabilizationWindowSeconds)，即负载下降后要持续一段时间才真正缩容。其作用是防止抖动：避免负载在阈值附近波动时副本数频繁增减 (thrashing)，保证服务稳定、减少 Pod 反复创建销毁的开销。

③ HPA 对降本的价值。HPA 让副本数随真实负载自动伸缩——低峰期仅保留 minReplicas=1 节省 CPU/内存与节点开销，高峰期自动扩容保障 SLA。相比按峰值固定容量部署，弹性伸缩显著提升资源利用率、降低云上成本，是云计算“按需付费、弹性供给”理念的直接体现。

第3章 第二部分：Spark 大数据分析

本部分在第一部分的 CCE 集群上，通过 Spark Operator 提交 PySpark 作业，对豆瓣电影数据集（OBS 存储，s3a 接口读取，约 6.7 万条记录）完成环境部署、数据清洗、SQL 统计分析与性能对比。

3.1 A-0：环境部署

3.1.1 Spark Operator 安装

使用离线 Helm Chart 安装 Spark Operator，并通过 `--set` 将控制器/Webhook 镜像指向 SWR group17（避免国内无法访问 ghcr.io）：

```
1 helm install spark-op ./spark-operator -n spark-operator \
2   --set image.registry=swr.cn-north-4.myhuaweicloud.com \
3   --set image.repository=group17/spark-operator --set image.tag=2.5.0 \
4   --set image.pullSecrets[0].name=default-secret \
5   --set spark.jobNamespaces={default}
```

Listing 3.1 安装 Spark Operator

3.1.2 关键问题：Executor 因 CPU 请求不足无法调度

首次提交作业时 Driver 持续 Pending，describe 显示 Insufficient cpu。原因是 2 个 2vCPU 节点被 CCE 系统组件占用了约 90% 的 CPU 请求额度（实际使用率仅个位数），而 Driver 默认按 `cores: 1` 申请 1000m CPU。解决办法是用 `coreRequest` 把 Kubernetes 的 CPU 请求与 Spark 逻辑核心解耦：Driver `coreRequest: "100m"`、Executor `coreRequest: "50m"`，并临时把 Web 应用缩容到 0 释放额度。调整后 Driver 与 2 个 Executor 均成功 Running。

3.1.3 WordCount 验证

提交 `wordcount.py`（读取 `s3a://group17/data/sample.txt`），Driver 状态变为 Completed，输出 Top10 词频，验证 Spark Operator + OBS s3a 读取链路正常：

```
1 Top 10 words: [('spark', 300), ('hello', 200), ('data', 150),
2 ('cloud', 100), ('is', 50), ('design', 50), ('world', 50), ...]
```

Listing 3.2 WordCount 输出

提交豆瓣分析作业后，`kubectl get pods -n default` 可见 1 个 Driver 与 2 个 Executor Pod（图 3-1）。

```
84688 E:\cCloud 0ms
>> kubectl get pods -n default -w
douban-analysis-driver      0/1    Pending    0    0s
douban-analysis-driver      0/1    Pending    0    0s
douban-analysis-driver      0/1    ContainerCreating  0    0s
douban-analysis-driver      1/1    Running    0    1s
doubananalysis-a0d1099eb1c04710-exec-1 0/1    Pending    0    0s
doubananalysis-a0d1099eb1c04710-exec-1 0/1    Pending    0    0s
doubananalysis-a0d1099eb1c04710-exec-1 0/1    ContainerCreating  0    0s
doubananalysis-a0d1099eb1c04710-exec-2 0/1    Pending    0    0s
doubananalysis-a0d1099eb1c04710-exec-2 0/1    Pending    0    0s
doubananalysis-a0d1099eb1c04710-exec-2 0/1    ContainerCreating  0    0s
doubananalysis-a0d1099eb1c04710-exec-2 1/1    Running    0    1s
doubananalysis-a0d1099eb1c04710-exec-1 1/1    Running    0    1s
```

图 3-1 Spark Driver + 2 个 Executor Pod 均 Running

3.2 A-1：数据清洗

用 PySpark 读取 OBS 上的 CSV（开启 `multiLine` 以正确解析含换行的 `summary` 字段），强制转换 `year/rating_score/rating_count` 等字段类型，打印 Schema 与前 5 行，原始记录 **67,132** 条。各字段缺失值比例如表 3-1（注意 `rating_score` 为 0 表示“评分人数不足、豆瓣未出分”，等价于缺失，缺失率高达 65%）。

表 3-1 各字段缺失值比例

字段	缺失比例	字段	缺失比例
year	3.00%	directors	13.15%
rating_score	65.06%	summary	28.56%
genres	8.02%	其余字段	0.00%

两种不同的缺失值处理策略：

- **year（时间维度关键字段）——dropna 删行：**年份缺失或越界（非 1920–2025）的记录无法用于时间趋势分析，直接删除。共删除 2,013 行，清洗后剩 **65,119** 行。
- **rating_score（分析核心指标）——fillna 均值填充：**直接删除 65% 的未评分电影会损失过多样本，故新增 `rating_filled` 列，用“已评分电影均分 6.55”填充缺失/为 0 的值，保留样本量用于计数类统计；而在评分高低相关查询中再过滤 `rating_score > 0`（共 22,724 部已评分电影）以保证均值不被填充值

干扰。

数值字段基本统计（describe）显示：year 均值 1998、rating_score（含 0）均值仅 2.41 而 rating_filled 均值 6.55、rating_count 最大约 199 万（《肖申克的救赎》）。清洗前后行数对比见图 3-2。

```
=====
A-1 clean: strategy1 year=dropna / strategy2 rating_score=fillna(mean)
=====
mean rating of rated movies (rating>0) = 6.55
rows before = 67132 -> after year-dropna = 65119 (dropped 2013)
rated movies (rating>0) = 22724
```

图 3-2 A-1 数据清洗：Schema、缺失值比例、清洗前后行数与统计

3.3 A-2: Spark SQL 统计分析

基于已评分电影视图，用 Spark SQL 完成 5 个查询，覆盖 GROUP BY 聚合、ORDER BY Top-N、时间维度趋势、窗口函数与 JOIN。

3.3.1 Q1：各电影类型数量与平均评分（GROUP BY）

表 3-2 各类型电影数量与平均评分 Top（部分）

类型	数量	均分	类型	数量	均分
剧情	11662	6.83	动画	1531	7.25
喜剧	5079	6.61	家庭	705	7.22
爱情	3610	6.50	传记	567	7.27
动作	3132	6.13	恐怖	2074	5.48

分析：剧情片数量最多（1.17 万部）且均分较高（6.83），是豆瓣电影的绝对主力；传记（7.27）、动画（7.25）、家庭（7.22）类均分最高，说明这类影片质量普遍较好、口碑稳定；而恐怖（5.48）、惊悚（5.90）类均分明显偏低，反映此类商业片良莠不齐、整体口碑一般。类型与评分的关系为内容生产提供了参考。

3.3.2 Q2：高分电影 Top10（ORDER BY Top-N）

筛选评分人数 ≥ 5 万的电影按评分降序，Top10 为：肖申克的救赎 (9.7)、霸王别姬 (9.6)、控方证人 (9.6)、阿甘正传 (9.5)、美丽人生 (9.5)、辛德勒的名单 (9.5)、人生果实 (9.5)、这个杀手不太冷 (9.4)、千与千寻 (9.4)、泰坦尼克号 (9.4)。

分析：加评分人数门槛可有效过滤”小众高分”噪声，得到的高分榜单与公认经典高度一致。榜单以欧美经典剧情片为主，《肖申克的救赎》以 9.7 分、近 200 万评分人数稳居榜首，体现了”高口碑 + 高参与度”的双重认可，说明评分与评分人数结合能更稳健地衡量影片影响力。

3.3.3 Q3：各年代电影数量与平均评分趋势（时间维度）

表 3-3 各年代电影数量与平均评分趋势

年代	电影数量	平均评分
1950s	584	7.41
1970s	1174	6.97
1990s	2382	7.09
2000s	6108	6.70
2010s	9885	6.08

分析：电影数量随年代快速增长，2010s 收录近万部，是 1990s 的 4 倍多，反映影视产业的爆发式扩张与数据收录的时效性偏向；但平均评分却从 1950s 的 7.41 持续下滑到 2010s 的 6.08。这一”量升质降”趋势，既可能源于近年商业片、网络电影大量涌入拉低均分，也存在经典老片经时间筛选后”幸存者偏差”导致老片均分偏高的因素。

3.3.4 Q4：每个类型评分最高的电影（窗口函数）

用 `ROW_NUMBER() OVER (PARTITION BY genre ORDER BY rating_score DESC)` 取每个类型评分第一的电影：剧情/犯罪 → 肖申克的救赎 (9.7)、爱情/同性 → 霸王别姬 (9.6)、悬疑 → 控方证人 (9.6)、喜剧 → 美丽人生 (9.5)、历史/战争 → 辛德勒的名单 (9.5)、动作 → 这个杀手不太冷 (9.4)、冒险 → 星际穿越 (9.3)。

分析：窗口函数在不缩减数据行的前提下完成”分组内排名”，相比 `GROUP BY` 更灵活。结果显示同一部经典片常在多个类型中夺冠（如肖申克的救赎同属剧情与犯罪），反映豆瓣的多标签体系；各类型头部影片评分都在 9.3 以上，是该类型的标杆之作。

3.3.5 Q5：高于本年代平均分的经典电影（JOIN）

将电影表与”各年代均分”子表 JOIN，筛选评分人数 ≥ 10 万且高于本年代均分的电影。结果如肖申克的救赎 (9.7 vs 年代均分 7.09)、千与千寻 (9.4 vs 6.70)、忠犬八公的故事 (9.4 vs 6.70) 等。

分析：JOIN 将明细数据与聚合指标关联，实现”相对基准”的衡量。这些影片不仅绝对分高，更显著高于同年代平均水平（高出 2–2.7 分），是真正”超越时代”的口碑标杆；该方法比单纯按绝对分排序更能体现影片在其时代背景下的相对卓越程度。

A-2 各查询结果截图见图 3-3 与图 3-4。

A-2 Q1 [GROUP BY] count + avg rating per genre, Top15			A-2 Q2 [ORDER BY Top-N] Top10 high-rated movies with >=50k votes				A-2 Q3 [time trend] movies count + avg rating per decade		
genre	cnt	avg_rating	title	year	rating_score	rating_count	decade	cnt	avg_rating
剧情	11662	6.83	肖申克的救赎	1994	9.7	1992071	1930	3	9.3
喜剧	5079	6.61	霸王别姬	1993	9.6	1486938	1940	231	7.35
爱情	3610	6.5	控方证人	1957	9.6	275167	1950	584	7.41
动作	3132	6.13	阿甘正传	1994	9.5	1509559	1960	624	7.3
惊悚	2691	5.9	美丽人生	1997	9.5	951526	1970	1174	6.97
犯罪	2208	6.65	辛德勒的名单	1993	9.5	775386	1980	1686	7.09
恐怖	2074	5.48	人生果实	2017	9.5	87871	1990	2382	7.09
unknown	1849	6.54	这个杀手不太冷	1994	9.4	1703344	2000	6108	6.7
悬疑	1568	6.42	千与千寻	2001	9.4	1573031	2010	9885	6.08
动画	1531	7.25	泰坦尼克号	1997	9.4	1458746	2020	47	6.04
冒险	1151	6.65							
科幻	1113	6.09							
奇幻	954	6.55							
家庭	705	7.22							
传记	567	7.27							

图 3-3 A-2 查询 Q1–Q3 结果

A-2 Q4 [window] top-rated movie per genre (ROW_NUMBER partition)				A-2 Q5 [JOIN] classics with >=100k votes above their decade average			
genre	title	rating_score	rating_count	title	year	rating_score	decade_avg
剧情	肖申克的救赎	9.7	1992071	肖申克的救赎	1994	9.7	7.09
犯罪	肖申克的救赎	9.7	1992071	霸王别姬	1993	9.6	7.09
同性	霸王别姬	9.6	1486938	控方证人	1957	9.6	7.41
悬疑	控方证人	9.6	275167	阿甘正传	1994	9.5	7.09
爱情	霸王别姬	9.6	1486938	美丽人生	1997	9.5	7.09
奇幻	夏日友人帐 第六季 特别篇 铃响的残株	9.5	10231	辛德勒的名单	1993	9.5	7.09
动画	夏日友人帐 第六季 特别篇 铃响的残株	9.5	10231	这个杀手不太冷	1994	9.4	7.09
历史	辛德勒的名单	9.5	775386	泰坦尼克号	1997	9.4	7.09
喜剧	美丽人生	9.5	951526	千与千寻	2001	9.4	6.7
纪录片	人生果实	9.5	87871	忠犬八公的故事	2009	9.4	6.7
家庭	唐朝庄园：2015圣诞特别篇	9.5	10961				
战争	辛德勒的名单	9.5	775386				
动作	这个杀手不太冷	9.4	1703344				
灾难	泰坦尼克号	9.4	1458746				
冒险	星际穿越	9.3	1075475				

图 3-4 A-2 查询 Q4–Q5 结果

3.4 A-3：性能对比与 Amdahl 分析

选取”按年代聚合平均评分”查询，分别用单机 Pandas 与 PySpark（1 个、2 个 Executor）实现，记录执行时间如表 3-4 与图 3-5。

表 3-4 Pandas vs PySpark 执行时间对比

方案	执行时间 (s)	相对 Pandas
Pandas（单机）	0.42	1.0×
PySpark（1 Executor）	13.58	32× 慢
PySpark（2 Executor）	15.90	38× 慢

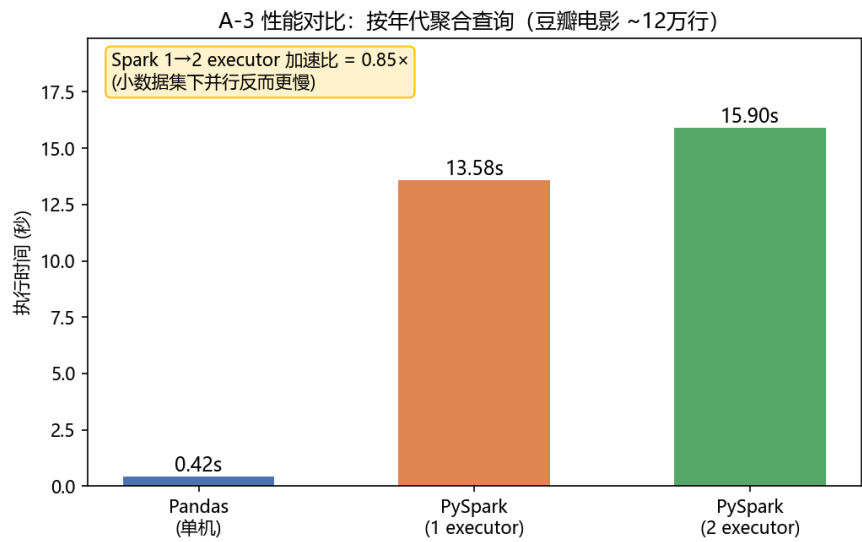


图 3-5 Pandas / PySpark(1) / PySpark(2) 执行时间对比

Amdahl 定律分析：本实验有两个反直觉但合理的现象。

①**为何 Spark 远慢于 Pandas？**数据集仅约 39 MB，属于”小数据”。Pandas 在单机内存中直接计算，几乎无额外开销；而 Spark 即便处理小数据，也要付出固定的”串行开销”：SparkContext/Executor 启动、与 OBS 建立 s3a 连接并通过网络读取、任务序列化与分发、shuffle 落盘与网络传输、结果回收到 Driver 等。按 Amdahl 定律 $S = \frac{1}{(1-p)+p/n}$ ，当问题规模小时，不可并行的串行部分 $(1-p)$ 占绝对主导，并行计算节省的那点时间远不足以抵消这些固定开销，故总时间被串行开销”锁死”在十几秒。

②**为何 2 个 Executor 反而比 1 个慢？**加速比仅 0.85×（不升反降）。因为数据被切分到 2 个 Executor 后，groupBy 聚合需要跨 Executor 进行 shuffle，引入了额外的网络通信、数据序列化/反序列化与协调开销；而每个 Executor 实际承担的计算量本就微乎其微（39 MB/2）。当通信开销 > 并行计算收益时，增加并行度反而拖慢整体——这正是 Amdahl 定律中”通信开销决定并行上限”的体现。

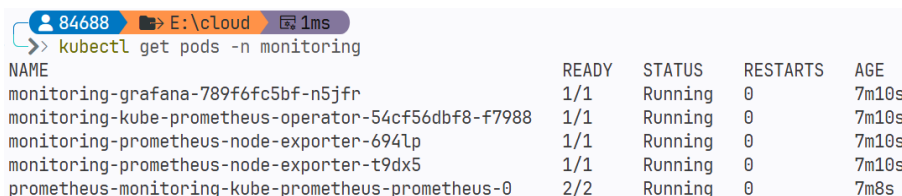
结论：Spark 的价值在于 **大规模数据**下的水平扩展能力。对本例的小数据集，单机 Pandas 是更优选择；只有当数据量增长到单机内存无法容纳、串行开销被巨大的并行计算收益摊薄时，Spark 的分布式优势才会显现。选择计算框架必须匹配数据规模。

第 4 章 附加题

4.1 附加题 1：监控系统 Prometheus + Grafana

4.1.1 部署

使用 kube-prometheus-stack 离线 Helm Chart 在 monitoring 命名空间部署 Prometheus + Grafana。为适配 2 节点小集群并规避镜像问题，自定义 values 做了如下处理：所有镜像指向 SWR group17；关闭 kubeStateMetrics 与 alertmanager 以省资源；覆盖 prometheusDefaultBaseImage 避免主容器仍从 quay.io 拉取；由于离线包未提供 grafana 的 k8s-sidecar 镜像，关闭 sidecar 并通过 additionalDataSources 显式声明 Prometheus 数据源。部署后所有组件 Running（图 4-1）。



NAME	READY	STATUS	RESTARTS	AGE
monitoring-grafana-789f6fc5bf-n5jfr	1/1	Running	0	7m10s
monitoring-kube-prometheus-operator-54cf56dbf8-f7988	1/1	Running	0	7m10s
monitoring-prometheus-node-exporter-694lp	1/1	Running	0	7m10s
monitoring-prometheus-node-exporter-t9dx5	1/1	Running	0	7m10s
prometheus-monitoring-kube-prometheus-prometheus-0	2/2	Running	0	7m8s

图 4-1 monitoring 命名空间组件全部 Running

4.1.2 Grafana Dashboard

在 Grafana 中构建仪表盘，展示节点 CPU 利用率折线图与各 Pod 内存使用柱状图（图 4-2）。关键 PromQL：

```
1 # 节点 CPU 利用率 (%)
2 100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) *
   100)
3 # 各 Pod 内存使用 (MiB)
4 topk(10, sum by(pod) (container_memory_working_set_bytes{namespace=~"
   default|monitoring"})) /1024/1024)
```

Listing 4.1 监控面板查询

4.1.3 Prometheus Pull 采集原理与指标含义

Pull 采集原理：与传统监控的 Push 模式不同，Prometheus 采用主动拉取（Pull）模型。被监控对象（node-exporter、kubelet/cAdvisor、各 Pod）暴露一个 /metrics HTTP 端点，以文本格式输出当前指标快照；Prometheus Server 根据配置

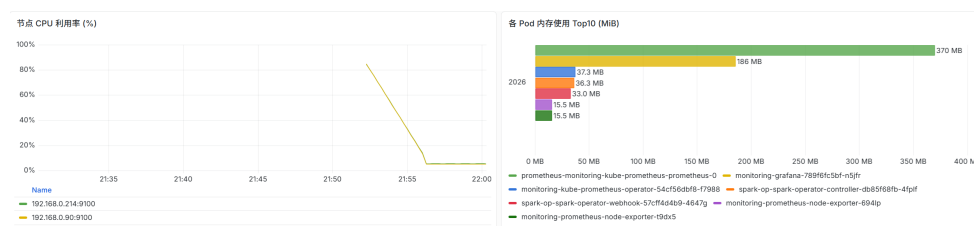


图 4-2 Grafana 仪表盘：节点 CPU 折线 + Pod 内存柱状

的 `scrape_interval`（本例约 30 s）周期性地向这些 `target` 发起 HTTP 请求拉取指标，打上时间戳后存入本地时序数据库（TSDB）。在 K8s 中，Prometheus Operator 通过 `ServiceMonitor` CRD 自动发现需要抓取的 `target`，无需手改配置。Pull 模型的优点是：服务端集中控制采集频率、`target` 健康状态一目了然（抓取失败即可判定宕机）、便于本地调试（直接 `curl /metrics`）。

至少 3 个指标的含义：

- `node_cpu_seconds_total`: **counter** 类型，节点各 CPU 核在各模式（`user/system/idle/iowait` 等）累计运行的秒数。对其 `idle` 模式求 `rate` 再用 100 减去，即得节点 CPU 利用率百分比。
- `container_memory_working_set_bytes`: **gauge** 类型，容器当前的工作集内存（活跃使用、不可被回收的内存），是判断 Pod 是否接近内存上限、是否会被 OOMKill 的关键指标。
- `kube_pod_status_phase/up`: `up` 为 Prometheus 对每个 `target` 抓取成功与否的探针（1= 成功，0= 失败），是判断监控目标存活的最基础指标。

4.2 附加题 2: CI/CD 流水线

4.2.1 流水线设计

基于 GitHub Actions 为第一部分应用搭建“代码提交 → 自动构建镜像 → 推送 SWR → 滚动更新 K8s Deployment”端到端流水线（`.github/workflows/deploy.yml`）。触发后依次执行：Checkout 代码 → 用 `git` 短 SHA 生成镜像 Tag → 登录 SWR → `docker build --provenance=false` 构建并推送后端镜像 → 用 `base64` 解码的 KubeConfig 配置 `kubectl` → `kubectl set image` 滚动更新 `backend` Deployment 并验证。仓库 Secrets 通过 GitHub API（PyNaCl sealed box 加密）自动写入 `SWR_AK`、`SWR_SK`（SWR 登录密钥）、`KUBE_CONFIG`（`base64` 编码的 `kubeconfig`）。实际运行结果：流水线 7 个步骤在约 60 s 内全部 **Passed**，后端 Deployment 镜像 Tag 由 `v1` 自动更新为 `git 短 SHA backend:ed09fde`，并滚动出一个运行新镜像的 Pod，验证了从

代码提交到集群部署的端到端自动化（GitHub 托管 runner 可直连华为云 SWR 与 CCE 公网 API）。各阶段 Passed 与镜像更新见图 4-3、4-4。

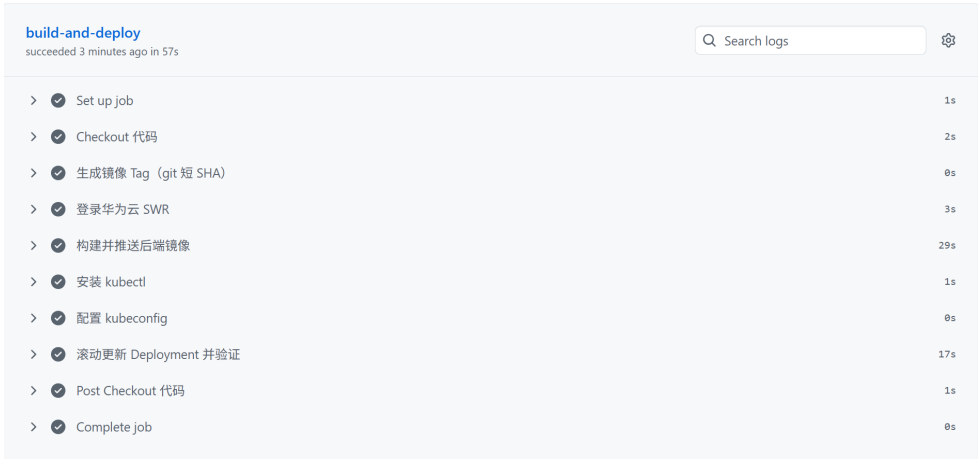


图 4-3 GitHub Actions 流水线各阶段全部 Passed



图 4-4 Deployment 镜像 Tag 已自动更新为 git SHA

4.2.2 持续集成 vs 持续部署与 GitOps

持续集成 (CI) vs 持续部署 (CD): CI 关注代码合并到构建验证这一段——每次提交自动触发编译、单元测试、构建镜像，尽早发现集成冲突，保证主干始终可构建；CD 关注从通过验证的产物到部署上线这一段——把构建好的镜像自动发布到测试/生产环境。简言之，CI 解决”代码能不能合到一起、构建出来”，CD 解决”构建好的东西怎么自动、可靠地上线”。本流水线的 build/push 属 CI，kubectl set image 滚动更新属 CD。

GitOps 核心理念：以 Git 仓库作为系统期望状态的唯一可信源（single source of truth）。所有基础设施与应用的声明式配置（YAML）都存在 Git 中，对系统的任何变更都通过 Git 提交（Pull Request）完成，再由自动化工具把集群实际状态持续向 Git 中声明的期望状态收敛。其价值在于：变更可审计、可回滚（git revert 即回滚）、环境可复现，把运维操作变成了代码评审与版本控制，极大提升了交付的可靠性与透明度。

4.3 附加题 3：分布式 AI 训练（PyTorch DDP）

4.3.1 实验设计

在 CCE 集群上用 PyTorch **DistributedDataParallel (DDP)** 训练经典 MNIST 手写数字识别 CNN（约 120 万参数），对比**单机**与 **2 个 Worker Pod 分布式**的训练时间。采用 CPU 友好的 **gloo** 通信后端，通过 `init_method='env://'` 从环境变量（`WORLD_SIZE/RANK/MASTER_ADDR`）完成 **rendezvous**；分布式模式下用 **headless Service** 让 Worker 通过稳定 DNS 找到 rank0 的会合端口。MNIST 数据集已在构建镜像时烘焙到 `/data`（节点无公网）。为公平对比强扩展性，每个 Worker 固定使用 1 个 CPU 线程：单机 = 1 Worker×1 线程，分布式 = 2 Worker×1 线程（合计 2 核）。

4.3.2 结果对比

表 4-1 单机 vs 分布式 DDP 训练时间对比（2 epochs，MNIST）

模式	Worker 数	训练时间 (s)	加速比
单机	1	148.27	1.00×
分布式 DDP	2	79.70	1.86×

实验结果：单机训练耗时 **148.27 s**、吞吐 809 samples/s；2 Worker 分布式（实际调度为每节点 1 个 Pod，真正跨节点）耗时 **79.70 s**、吞吐 1506 samples/s，**加速比 1.86×**（扩展效率约 93%）。两种模式最终测试准确率分别为 98.80% 与 98.69%，几乎相同，说明分布式训练在近乎线性提速的同时保持了模型精度。训练 Pod 运行情况与 `##RESULT##` 输出见图 4-5、4-6。

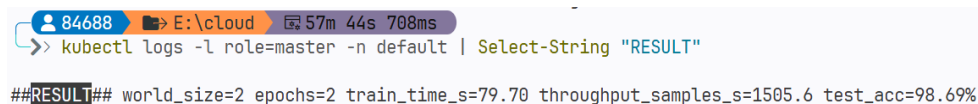
84688

E:\cloud

2ms

```
>> kubectl get pods -l app=ddp -n default -w
ddp-master-jwfdw 0/1 Pending 0 0s
ddp-master-jwfdw 0/1 Pending 0 0s
ddp-master-jwfdw 0/1 ContainerCreating 0 0s
ddp-worker-f2kkh 0/1 Pending 0 0s
ddp-worker-f2kkh 0/1 Pending 0 0s
ddp-master-jwfdw 1/1 Running 0 1s
ddp-worker-f2kkh 0/1 Pending 0 26s
ddp-worker-f2kkh 0/1 ContainerCreating 0 26s
ddp-worker-f2kkh 1/1 Running 0 27s
```

图 4-5 分布式训练 master + worker Pod 运行



```
84688 E:\cloud 57m 44s 708ms  
> kubectll logs -l role=master -n default | Select-String "RESULT"  
##RESULT## world_size=2 epochs=2 train_time_s=79.70 throughput_samples_s=1505.6 test_acc=98.69%
```

图 4-6 单机与分布式训练时间与准确率对比输出

4.3.3 AllReduce 梯度同步机制

DDP 属于**数据并行**：完整的模型副本部署在每个 Worker 上，训练数据被 DistributedSampler 切分给不同 Worker，各自用本地 mini-batch 做前向与反向传播，得到**本地梯度**。关键在于反向传播后、参数更新前，所有 Worker 必须把各自的梯度**聚合求平均**，使每个副本以相同的“全局平均梯度”更新参数，从而保持各副本模型始终一致——这一步由 **AllReduce** 集合通信完成。

AllReduce 把“归约 (Reduce, 如求和)”与“广播 (Broadcast)”合二为一：所有进程的梯度张量按元素相加后，结果再分发回所有进程。高效实现常用 **Ring-AllReduce**：N 个 Worker 组成环，梯度切成 N 份，经过 $2(N-1)$ 步的“scatter-reduce + all-gather”，每步只与相邻节点通信，使通信量与 N 基本无关、带宽利用最优。PyTorch DDP 还会在反向传播过程中将**梯度分桶 (bucket)** 并与**计算重叠**，边算边通信以隐藏延迟。

4.3.4 数据并行 vs 模型并行

数据并行 (Data Parallelism)：每个设备持有**完整模型**、处理不同数据分片，通过 AllReduce 同步梯度。适合模型能放进单卡显存、但数据量大的场景，是目前最常用的并行方式（本实验即属此类）。**模型并行 (Model Parallelism)**：把**单个模型**的不同层/张量切分到**不同设备**，一份数据依次/并行流经各设备。适合模型大到单卡放不下的场景（如超大语言模型），但设备间需传递中间激活值、通信更频繁、实现更复杂。二者并非互斥，超大模型常用“数据并行 + 张量并行 + 流水线并行”的混合策略。

4.3.5 批判性思考

本实验取得了 **1.86x** 的加速比（扩展效率约 93%），相当接近理想的 2x。这与 A-3 中 Spark “2 executor 反而更慢”形成鲜明对比，原因正是**计算-通信比**不同：MNIST CNN 每个 step 的前向/反向计算（约 120 万参数）在 CPU 上耗时可观，而每步的梯度 AllReduce 只需同步同样大小的梯度张量，**计算时间远大于通信时间**，因此并行计算的收益足以摊薄通信开销，扩展性接近线性；而 A-3 的聚合查询数据极小、计算极轻，通信开销占了主导，于是并行不升反降。

未达到完美 2x 的约 7% 差距, 来自每步 gloo over TCP 的 AllReduce 通信、进程会合、barrier 同步与数据加载等无法并行的串行部分 (Amdahl 定律的 $1-p$)。由此可得出一个重要判断: **分布式训练并非“越多节点越快”, 其扩展上限由计算-通信比决定**。若改用 GPU + 高速互联 (NVLink/RDMA), 单步计算更快、AllReduce 更高效, 但要维持高扩展效率往往还需增大 batch size 以保证每步计算量。理解并优化“计算-通信比”, 是设计高效分布式训练系统的核心, 也是本实验给我的最大启发。

第 5 章 总结与收获

本次课程设计在华为云上完整走通了「容器化 → K8s 编排 → 大数据计算 → 运维监控 → 持续交付 → 分布式训练」的云原生全链路，从把概念落到了真实生产环境。

对云计算技术的新认识。其一，声明式与不可变带来的确定性。整个平台用 YAML 声明期望状态，`kubectl apply` 后由控制器持续调谐，副本数、镜像版本、配置都可版本化、可复现；多阶段构建产出的不可变镜像（后端仅 202 MB）配合 SWR 私有仓库，使「一次构建、处处运行」成为现实。其二，关注点分离。`ConfigMap/Secret` 把配置与镜像解耦，`PVC` 把数据生命周期与 `Pod` 解耦（删除并重建 Redis `Pod` 后 `testkey` 仍在），`Service/ELB` 把访问入口与具体 `Pod` 解耦，每一层都可独立演进。其三，弹性即成本。`HPA` 让后端副本在 1~4 之间随真实 CPU 负载自动伸缩，低峰省钱、高峰保障 SLA，直接体现「按需付费」。其四，分布式的代价与收益。`Spark` 与 `PyTorch DDP` 的实验让我直观看到并行加速并非线性——通信、序列化与数据规模决定了 Amdahl 定律中的串行瓶颈。

主要挑战与解决。实践中遇到并解决了一系列真实工程问题：①国内网络无法直连 Docker Hub / ghcr.io / quay.io，通过 daocloud 镜像代理拉取后重打 Tag 推送到 SWR 解决；② Docker 29.x 默认 OCI manifest 与 SWR 不兼容，构建加 `--provenance=false`；③集群默认未启用 `metrics-server` 导致 `HPA` 无数据，手动部署并加 `--kubelet-insecure-tls`；④ 2 节点（各 2 vCPU/8 GiB）资源紧张，运行 `Spark`/训练作业前先把 Web 应用副本缩容、并用 `coreRequest` 把 `Spark Executor` 的 K8s CPU 请求与逻辑核心解耦，规避 `FailedScheduling`；⑤ `ConfigMap` 的 `subPath` 挂载不会热更新，通过删除 `Pod` 重建加载新配置。这些“坑”恰恰是云上工程最有价值的经验。

总体而言，本设计不仅完成了全部规定任务与三个附加专题，更让我体会到云原生体系中抽象层层叠加、控制器持续调谐、资源弹性供给的设计哲学，对今后从事云计算与大数据相关开发打下了坚实基础。

第 6 章 附录

6.1 代码仓库

本课程设计的全部源码（Dockerfile、应用代码、K8s 清单、Spark 作业、监控 values、CI/CD 工作流、PyTorch DDP 训练脚本、本报告 L^AT_EX 源码）均托管于 GitHub:

<https://github.com/YCYQLG/CC>

6.2 关键资源清单

表 6-1 部署的 Kubernetes 资源一览

类别	名称	说明
Deployment	backend / redis / frontend	后端副本 2、Redis(PVC)、前端 Nginx
Service	backend-svc (LoadBalancer)	绑定公网 ELB 1.94.238.112
Service	redis-svc / backend-internal	ClusterIP 集群内访问
ConfigMap	backend-config / frontend-nginx-conf	注入配置 / 卷挂载
Secret	redis-secret / obs-credentials	密码与 OBS 凭据 (base64)
PVC	redis-data-pvc	10Gi csi-disk(EVS), Bound
HPA	backend-hpa	min1/max4/CPU 60%
SparkApplication	douban-analysis 等	PySpark 数据分析作业

6.3 镜像清单 (SWR group17)

swr.cn-north-4.myhuaweicloud.com/group17/ 下推送了:backend:v1、frontend:v1、redis:7-alpine、pyspark:v9、spark-operator:2.5.0、pytorch-ddp:v1、metrics-server:v0.7.2 以及监控套件 (grafana、prometheus、alertmanager、node-exporter 等) 共十余个镜像, 均设为可被 CCE 同账号节点拉取。